

Soft Robotic Finger: Progress So Far

Goal

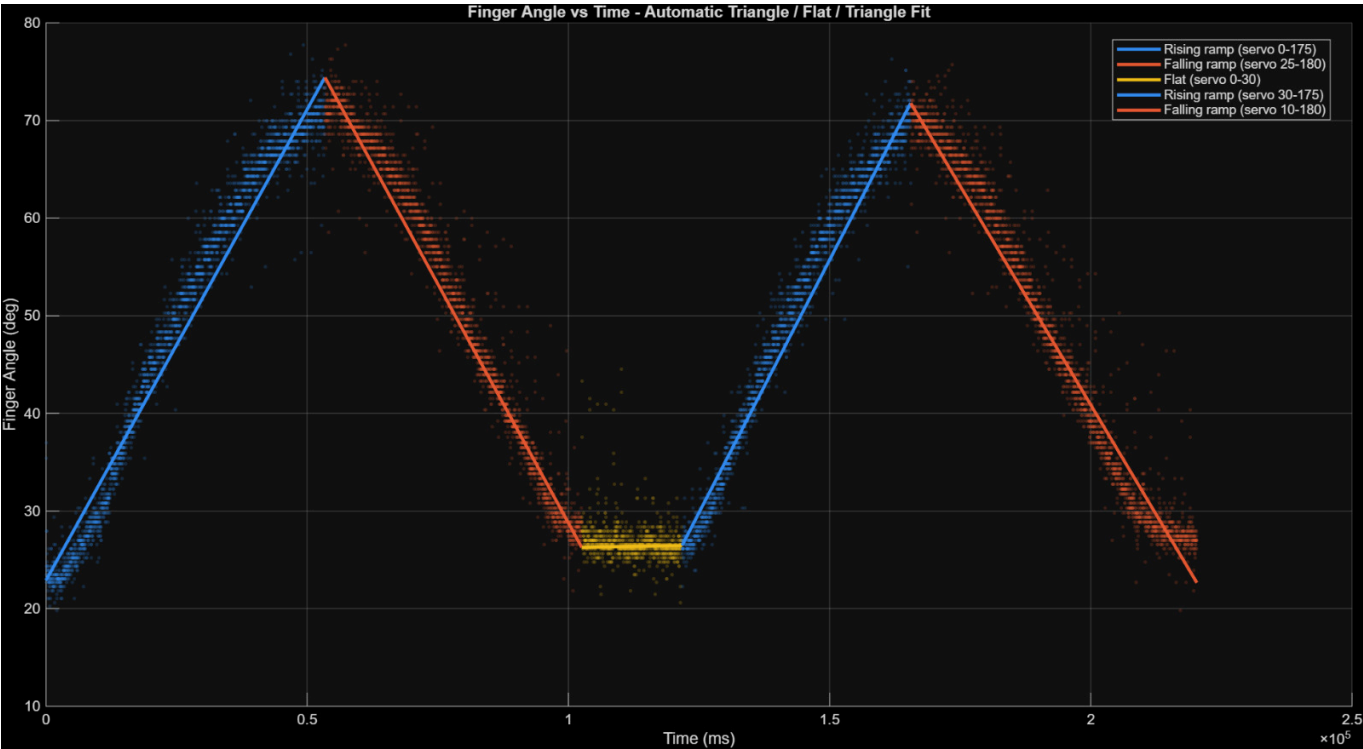
Control the bending angle of a soft robotic finger from an Arduino, using a neural network.

Setup

- A silicone soft finger with a tendon (cable) running through it.
- A servo motor pulls the tendon. When the servo turns, the finger bends.
- A sensor measures the finger angle.
- An Arduino drives the servo and reads the sensor. The data is logged to a PC.

What we did

We ran an open-loop test. The Arduino moved the servo slowly up and down (0° to 180°) in a triangle pattern for about 220 seconds. We recorded the servo angle and the finger angle over time. Then we fitted a straight line to each up and down segment.



```
--- Segment Summary (chronological) ---
[RISE] t=      0- 53389 ms : finger = 0.3084*servo + 22.8850 (R^2=0.9680)
[FALL] t= 53410- 102561 ms : finger = 0.3085*servo + 17.9885 (R^2=0.9668)
[FLAT ] t= 102582- 121733 ms : finger ~= 26.83 deg (constant)
[RISE] t= 121754- 165506 ms : finger = 0.3286*servo + 16.2437 (R^2=0.9718)
[FALL] t= 165527- 220295 ms : finger = 0.2859*servo + 19.7910 (R^2=0.9627)
```

Segment	Time (s)	Servo range	Fit: finger angle (deg) =	R ²
Rise	0 – 53.4	0 → 175	0.3084 × servo + 22.89	0.968

Segment	Time (s)	Servo range	Fit: finger angle (deg) =	R ²
Fall	53.4 – 102.6	180 → 25	$0.3085 \times \text{servo} + 17.99$	0.967
Flat	102.6 – 121.7	0 – 30	≈ 26.8 (constant)	—
Rise	121.8 – 165.5	30 → 175	$0.3286 \times \text{servo} + 16.24$	0.972
Fall	165.5 – 220.3	180 → 10	$0.2859 \times \text{servo} + 19.79$	0.963

What we found

1. **Near-linear response.** The finger bends about 0.3° for every 1° of servo rotation. A straight line explains about 97% of the behaviour.
2. **Working range.** The finger moves between about 23° and 76° .
3. **Dead zone.** Between servo 0° and 30° , the finger does not move (it stays at about 27°) because the tendon is slack. Usable servo range is about 30° to 180° .
4. **Hysteresis.** The finger angle depends on whether the servo is moving up or down. At the same servo angle, the difference is up to about 5° .
5. **Drift.** The second cycle is not the same as the first. The slopes change by about 7%, and the resting angle moved from 23° to 27° .
6. **Noise.** The measured angle scatters about $\pm 2\text{--}3^\circ$ around the fitted lines.

What this means for control

- A simple straight-line map from target angle to servo angle already gets close, but it is off by a few degrees because of the dead zone, hysteresis and drift.
- A neural network can learn the dead zone and the up/down difference.
- Drift changes over time, so the network alone is not enough. Sensor feedback is also needed.

Planned controller

1. A small neural network (2 inputs, 8 neurons, 1 output) takes the target angle and the direction of motion, and gives the servo angle.
2. A PI feedback loop on the measured angle corrects the remaining error.
3. The servo command is limited to $30^\circ\text{--}180^\circ$ to avoid the dead zone.

The network has about 33 numbers, so it fits easily on an Arduino Uno. The training script and the Arduino code are drafted but not yet tested on the finger.

Next

- Collect more data: different speeds, different amplitudes, step inputs, and a load on the fingertip.
- Train the network and compare it against the straight-line fit.
- Run the network with PI feedback on the real finger and measure the tracking error.